

LangGraph

Graph-Based Orchestration Framework for Stateful AI Agents

Introduction

LangGraph is an open-source framework, built on top of LangChain, for building stateful, multi-step LLM applications as explicit directed graphs rather than implicit prompt chains. Instead of stringing prompts together and hoping the control flow works out, LangGraph asks you to define a typed State schema, register Node functions that read and transform that state, and connect those nodes with Edges — either fixed (always the same next step) or conditional (chosen dynamically by a router function). The resulting graph is compiled into a runnable object that can be invoked like a regular function.

LangGraph's core strength is making agent behaviour explicit and inspectable: branching logic, tool calls, memory, and multi-agent hand-offs are all visible as nodes and edges in the graph rather than buried inside a single giant prompt. Built-in checkpointing (e.g., `SqliteSaver`) adds durable, cross-session memory with very little code, making LangGraph well suited for chatbots, support routers, ReAct tool-using agents, and multi-agent systems that need to remember context across runs.

Components

Component	Description
StateGraph	The graph builder class used to register nodes and edges.
State (TypedDict)	A typed dictionary defining every field that flows through the graph.
Node	A Python function, registered via <code>add_node()</code> , that reads and updates state.
Fixed Edge	<code>add_edge()</code> — an unconditional, always-the-same connection between two nodes.
Conditional Edge	<code>add_conditional_edges()</code> — branches to a node chosen by a router function.
Entry / Finish Point / END	Marks where graph execution starts and where it may terminate.
<code>compile()</code>	Locks the graph structure into an executable object; optionally attaches a checkpointer.
Checkpointers (SqliteSaver)	Persists state to SQLite after every node, keyed by <code>thread_id</code> .
<code>add_messages</code> Reducer	Makes a message-list state field append instead of overwrite.
<code>invoke()</code>	Runs the compiled graph once with a given initial state and returns the final state.

Syntax

```
# 1. Define the State
from typing import TypedDict
from langgraph.graph import StateGraph

class AgentState(TypedDict):
    field_in: str
    field_out: str

# 2. Define a Node
def my_node(state: AgentState) -> AgentState:
    state["field_out"] = do_something(state["field_in"])
    return state

# 3. Build the graph
builder = StateGraph(AgentState)
builder.add_node("node_name", my_node)
builder.set_entry_point("node_name")
builder.set_finish_point("node_name")

# 4a. Fixed edge (always go from A to B)
builder.add_edge("node_a", "node_b")

# 4b. Conditional edge (branch based on a router function)
def router(state: AgentState) -> str:
    return "branch_a" if state["field_out"] == "x" else "branch_b"

builder.add_conditional_edges("node_name", router,
    {"branch_a": "node_a", "branch_b": "node_b"})

# 5. Compile and run
graph = builder.compile()
result = graph.invoke({"field_in": "hello", "field_out": ""})

# 6. Compile with persistent memory (SQLite checkpointer)
from langgraph.checkpoint.sqlite import SqliteSaver
import sqlite3
conn = sqlite3.connect("memory.db", check_same_thread=False)
graph = builder.compile(checkpointer=SqliteSaver(conn))
result = graph.invoke(initial_state, config={"configurable": {"thread_id": "user_001"}})
```

Learning Outcomes

- Build a LangGraph StateGraph using State, Node, fixed edges, and conditional edges.
- Integrate a local LLM (Qwen 2.5 via Ollama) for classification, extraction, and generation.
- Implement the ReAct tool-calling pattern (Reason -> Act -> Respond).
- Add session memory and durable SQLite-based persistent memory to an agent.
- Design multi-agent systems using the Supervisor/router pattern with knowledge grounding.
- Debug common agent/graph wiring issues such as routing and state-field errors.

Software Requirements

Requirement	Detail
Python Version	3.10 or higher (3.11 recommended)
Core Packages	langgraph, langchain-core, langchain-ollama
Persistence (Labs 6-7)	langgraph-checkpoint-sqlite (provides SqliteSaver)
Other Libraries	rich (console output), sqlite3 (stdlib, persistence)
LLM Backend	Ollama running locally, serving qwen2.5:3b
API Key	None required — fully local via Ollama
Install Command	pip install langgraph langchain-core langchain-ollama rich langgraph-checkpoint-sqlite

Lab Programs

Lab 1: Foundations of LangGraph — Single-Node StateGraph

Source Code

```

from typing import TypedDict
from langgraph.graph import StateGraph

# -----
# STEP 1: Define the State
# -----

class AgentState(TypedDict):
    name: str      # Input: who is being greeted
    greeting: str  # Output: filled in by the node below

# -----
# STEP 2: Define a Node (a function that does work)
# -----

def generate_greeting(state: AgentState) -> AgentState:
    """
    Node: Greeting Generator
    Reads 'name' from state, writes a greeting back.
    """
    name = state["name"]
    state["greeting"] = f"Welcome {name} to LangGraph Training!"
    return state

# -----
# STEP 3: Build the Graph

```

```

# -----

builder = StateGraph(AgentState)

# Register the node -- give it a name and a function
builder.add_node("greet", generate_greeting)

# Entry point = where execution starts
builder.set_entry_point("greet")

# Finish point = where execution ends
builder.set_finish_point("greet")

# Compile = lock in the graph structure, ready to run
graph = builder.compile()

# -----
# STEP 4: Run the Graph
# -----
if __name__ == "__main__":
    result = graph.invoke({"name": "Shyni"})
    print(result["greeting"])

    for name in ["Alice", "Bob", "VIT Student"]:
        output = graph.invoke({"name": name})
        print(output["greeting"])
# =====

```

Sample Input / Output

```

graph.invoke({"name": "Shyni"})

--- Output ---
Welcome Shyni to LangGraph Training!

```

Key Concepts Covered

Concept	Description
TypedDict State	Defines the data flowing through the graph.
Node	A function that reads/updates state.
Entry/Finish Point	Marks where the graph starts and stops.

Lab 2: AI-Powered Customer Support Router

Source Code

```
# =====
# LAB 3 -- AI-Powered Customer Support Router
# =====

from typing import TypedDict
from langgraph.graph import StateGraph
from langchain_ollama import ChatOllama
from rich.console import Console
from rich.panel import Panel

console = Console()

# -----
# STEP 1: Connect to local Qwen model via Ollama
# -----

llm = ChatOllama(
    model="qwen2.5:3b",
    temperature=0
)

# -----
# STEP 2: Define the State
# -----

class SupportState(TypedDict):
    query: str      # Customer's original question
    category: str    # Classified by LLM: billing / technical / general
    answer: str      # Response from the routed department agent

# -----
# STEP 3: Classification Node (uses LLM)
# -----

def classify(state: SupportState) -> SupportState:
    """
    Uses Qwen to classify the customer query into a category.
    """
    prompt = f"""You are a customer support classifier.

    Classify the following customer query into exactly ONE category.

    Query: {state['query']}

    Rules:
    - Reply with ONLY one word: billing, technical, or general
    - Do NOT add explanations, punctuation, or extra words
    - billing = payment, invoice, refund, subscription, charges
    - technical = error, bug, crash, not working, installation, login issues
    - general = hours, location, product info, policies, other
    """

    result = llm.invoke(prompt)
    raw = result.content.strip().lower()

    # Safety: if LLM returns something unexpected, default to general
    valid_categories = {"billing", "technical", "general"}
    state["category"] = raw if raw in valid_categories else "general"
```

```

    console.print(f"[bold cyan]Classified as: {state['category'].upper()}[/bold cyan]")
    return state

# -----
# STEP 4: Department Agent Nodes
# -----

def billing_agent(state: SupportState) -> SupportState:
    state["answer"] = (
        "BILLING SUPPORT\n"
        "Your query has been received by our Billing Team.\n"
        "Typical issues handled: payment failures, invoice disputes, refunds.\n"
        "Expected resolution: 24-48 hours.\n"
        "Reference: " + state["query"][:30] + "..."
    )
    return state

def technical_agent(state: SupportState) -> SupportState:
    state["answer"] = (
        "TECHNICAL SUPPORT\n"
        "Your query has been assigned to our Technical Team.\n"
        "Please provide: device type, OS version, error message screenshot.\n"
        "Expected resolution: 2-4 hours.\n"
        "Reference: " + state["query"][:30] + "..."
    )
    return state

def general_agent(state: SupportState) -> SupportState:
    state["answer"] = (
        "GENERAL SUPPORT\n"
        "Thank you for contacting us.\n"
        "Our support team will get back to you shortly.\n"
        "Operating hours: Mon-Fri, 9 AM - 6 PM IST.\n"
        "Reference: " + state["query"][:30] + "..."
    )
    return state

# -----
# STEP 5: Router Function
# -----

def router(state: SupportState) -> str:
    """
    Returns the category string that maps to the next node.
    LangGraph uses this return value to pick which branch to follow.
    """
    return state["category"]

# -----
# STEP 6: Build the Graph with Conditional Edges
# -----

builder = StateGraph(SupportState)

builder.add_node("classify", classify)
builder.add_node("billing", billing_agent)
builder.add_node("technical", technical_agent)
builder.add_node("general", general_agent)

builder.add_conditional_edges(
    "classify",

```

```

    router,
    {
        "billing": "billing",
        "technical": "technical",
        "general": "general",
    }
)

builder.set_entry_point("classify")

# All three department nodes are finish points
builder.set_finish_point("billing")
builder.set_finish_point("technical")
builder.set_finish_point("general")

graph = builder.compile()

# -----
# STEP 7: Demo with Diverse Queries
# -----

test_queries = [
    "My payment failed yesterday and I was charged twice",
    "The app crashes every time I try to login from my iPhone",
    "What are your office hours?",
    "I need a refund for my last invoice",
    "How do I reset my password? The reset email never arrives",
    "Do you offer student discounts?",
]

if __name__ == "__main__":
    console.print("\n[bold magenta]=== AI Customer Support Router ===[/bold magenta]\n")

    for query in test_queries:
        console.print(f"\n[bold white]Customer:[/bold white] {query}")

        result = graph.invoke({
            "query": query,
            "category": "",
            "answer": ""
        })

        console.print(Panel(
            result["answer"],
            title=f"[bold green]Response ({result['category'].upper()})[/bold green]",
            border_style="green"
        ))

    console.print("\n[bold yellow]All queries routed successfully.[/bold yellow]")

```

Sample Input / Output

"My payment failed yesterday and I was charged twice"

--- Output ---

Classified as: BILLING

BILLING SUPPORT – Your query has been received by our
Billing Team. Expected resolution: 24-48 hours.

Key Concepts Covered

Concept	Description
LLM Classification	Constrains LLM output to one of a fixed set of labels.
Conditional Edges	Branch to a different node based on a router function's output.
ChatOllama	LangChain wrapper to call the local Qwen model.

Lab 3: Multi-Step Sequential Workflow — University Admission

Source Code

```
# =====
# LAB 2 -- Multi-Step Workflow: University Admission Processing
# =====

from typing import TypedDict
from langgraph.graph import StateGraph
from rich.console import Console
from rich.table import Table
import random
import datetime

console = Console()

# -----
# STEP 1: Define the State
# -----

class StudentState(TypedDict):
    name: str
    age: int
    marks: float          # Entrance exam marks (out of 100)
    student_id: str       # Generated by create_id node
    department: str       # Assigned by assign_department node
    scholarship: str      # Determined by check_scholarship node
    message: str          # Final welcome message

# -----
# STEP 2: Define Nodes
# -----

def create_id(state: StudentState) -> StudentState:
    """
    Node 1: Generate a unique student ID.
    Pattern: VIT-YEAR-RANDOM_NUMBER
    """
    year = datetime.datetime.now().year
    number = random.randint(1000, 9999)
    state["student_id"] = f"VIT-{year}-{number}"
    console.print(f"[cyan]-> ID Generated: {state['student_id']}[/cyan]")
    return state

def assign_department(state: StudentState) -> StudentState:
    """
    Node 2: Assign department based on entrance marks.

    Marks 90+      -> AI & Data Science
    Marks 75-89    -> Computer Science Engineering
    Marks 60-74    -> Electronics & Communication
    Marks below    -> Mechanical Engineering
    """
    marks = state["marks"]

    if marks >= 90:
        state["department"] = "AI & Data Science"
    elif marks >= 75:
        state["department"] = "Computer Science Engineering"
    elif marks >= 60:
        state["department"] = "Electronics & Communication"
    else:
        state["department"] = "Mechanical Engineering"
```

```

        console.print(f"[green]-> Department Assigned: {state['department']}[/green]")
        return state

def check_scholarship(state: StudentState) -> StudentState:
    """
    Node 3: Determine scholarship eligibility.

    Marks 95+ -> Full Scholarship (100%)
    Marks 85+ -> Partial Scholarship (50%)
    Marks 75+ -> Merit Award (25%)
    Below 75 -> No Scholarship
    """
    marks = state["marks"]

    if marks >= 95:
        state["scholarship"] = "Full Scholarship (100%)"
    elif marks >= 85:
        state["scholarship"] = "Partial Scholarship (50%)"
    elif marks >= 75:
        state["scholarship"] = "Merit Award (25%)"
    else:
        state["scholarship"] = "No Scholarship"

    console.print(f"[yellow]-> Scholarship: {state['scholarship']}[/yellow]")
    return state

def generate_welcome(state: StudentState) -> StudentState:
    """
    Node 4: Compose the final welcome message using all state fields.
    This node only READS -- it doesn't call any external service.
    """
    state["message"] = (
        f"Dear {state['name']},\n"
        f"Welcome to VIT University!\n"
        f"Student ID    : {state['student_id']}\n"
        f"Department     : {state['department']}\n"
        f"Scholarship    : {state['scholarship']}\n"
        f"We look forward to your journey with us."
    )
    return state

# -----
# STEP 3: Build the Graph
# -----

builder = StateGraph(StudentState)

builder.add_node("create_id", create_id)
builder.add_node("assign_department", assign_department)
builder.add_node("check_scholarship", check_scholarship)
builder.add_node("generate_welcome", generate_welcome)

# Linear chain: each node feeds into the next
builder.add_edge("create_id", "assign_department")
builder.add_edge("assign_department", "check_scholarship")
builder.add_edge("check_scholarship", "generate_welcome")

builder.set_entry_point("create_id")
builder.set_finish_point("generate_welcome")

graph = builder.compile()

```

```
# -----
# STEP 4: Run with Multiple Students
# -----

def print_result(result: StudentState):
    table = Table(title=f"Admission Result -- {result['name']}", style="bold")
    table.add_column("Field", style="cyan", width=20)
    table.add_column("Value", style="white", width=40)
    table.add_row("Student ID", result["student_id"])
    table.add_row("Department", result["department"])
    table.add_row("Scholarship", result["scholarship"])
    console.print(table)
    console.print(f"\n[bold green]{result['message']}[/bold green]\n")
    console.print("-" * 60)

if __name__ == "__main__":
    students = [
        {"name": "Ananya", "age": 18, "marks": 97.0, "student_id": "", "department": "",
         "scholarship": "", "message": ""},
        {"name": "Ravi", "age": 19, "marks": 82.5, "student_id": "", "department": "",
         "scholarship": "", "message": ""},
        {"name": "John", "age": 18, "marks": 63.0, "student_id": "", "department": "",
         "scholarship": "", "message": ""},
        {"name": "Priya", "age": 18, "marks": 55.0, "student_id": "", "department": "",
         "scholarship": "", "message": ""},
    ]

    for student in students:
        console.print(f"\n[bold magenta]Processing: {student['name']} (Marks:
{student['marks']})[/bold magenta]")
        result = graph.invoke(student)
        print_result(result)
```

Sample Input / Output

```
{"name": "Ananya", "marks": 97.0}

--- Output ---
Student ID : VIT-2026-4821
Department : AI & Data Science
Scholarship : Full Scholarship (100%)
```

Key Concepts Covered

Concept	Description
Fixed Edges	add_edge() always connects the same two nodes in sequence.
Sequential Pipeline	Each node enriches state for the next node to use.
Rule-Based Logic	Plain if/elif business rules — no LLM required.

Lab 4: ReAct Agent with Tool Calling

Source Code

```
# =====
# LAB 4 -- ReAct Agent with Tool Calling
# =====

from typing import TypedDict, Optional
from langgraph.graph import StateGraph
from langchain_ollama import ChatOllama
from rich.console import Console
from rich.panel import Panel
import math
import json

console = Console()

llm = ChatOllama(model="qwen2.5:3b", temperature=0)

# -----
# STEP 1: State
# -----

class AgentState(TypedDict):
    query: str
    tool_name: str          # Which tool the LLM chose
    tool_input: str         # Input to pass to the tool
    tool_result: str        # Output from the tool
    final_answer: str       # Human-readable final response

# -----
# STEP 2: Tool Implementations
# -----

def calculator_tool(expression: str) -> str:
    """
    Safe mathematical expression evaluator.
    Supports: +, -, *, /, **, sqrt, sin, cos, log, pi, e
    """
    try:
        expression = expression.replace("^", "**")
        expression = expression.replace("sqrt(", "math.sqrt(")
        expression = expression.replace("sin(", "math.sin(")
        expression = expression.replace("cos(", "math.cos(")
        expression = expression.replace("log(", "math.log(")
        expression = expression.replace("pi", str(math.pi))
        expression = expression.replace("e", str(math.e))

        allowed = set("0123456789+-*/.() math")
        if not all(c in allowed or c.isalpha() for c in expression):
            return "Error: Invalid characters in expression"

        result = eval(expression, {"__builtins__": {}, "math": math})
        return f"{result:.4f}" if isinstance(result, float) else str(result)
    except Exception as ex:
        return f"Calculator Error: {str(ex)}"

def weather_tool(city: str) -> str:
    """
    Mock weather tool -- simulates a real weather API response.
    In production, replace with: requests.get(f'https://api.openweathermap.org/...')
    """
```

```

mock_weather_db = {
    "chennai": {"temp": 34, "humidity": 78, "condition": "Partly Cloudy", "wind": "18
km/h"},
    "coimbatore": {"temp": 29, "humidity": 65, "condition": "Clear Sky", "wind": "12
km/h"},
    "mumbai": {"temp": 31, "humidity": 85, "condition": "Humid and Hazy", "wind": "22
km/h"},
    "delhi": {"temp": 28, "humidity": 55, "condition": "Sunny", "wind": "10
km/h"},
    "bangalore": {"temp": 25, "humidity": 60, "condition": "Light Drizzle", "wind": "8
km/h"},
    "hyderabad": {"temp": 32, "humidity": 58, "condition": "Mostly Sunny", "wind": "15
km/h"},
    "london": {"temp": 12, "humidity": 80, "condition": "Overcast", "wind": "25
km/h"},
    "new york": {"temp": 18, "humidity": 62, "condition": "Partly Cloudy", "wind": "20
km/h"},
}

city_lower = city.lower().strip()
if city_lower in mock_weather_db:
    w = mock_weather_db[city_lower]
    return (
        f"Weather in {city.title()}: {w['condition']}, "
        f"Temp: {w['temp']}C, Humidity: {w['humidity']}%, "
        f"Wind: {w['wind']}"
    )
    return f"Weather data not available for '{city}'. Try: Chennai, Coimbatore, Mumbai, Delhi,
Bangalore."

def unit_converter_tool(query: str) -> str:
    """
    Converts common units.
    Query format examples:
    '100 km to miles'
    '50 celsius to fahrenheit'
    '5 kg to pounds'
    """
    query = query.lower()

    try:
        if "celsius" in query and "fahrenheit" in query:
            val = float(''.join(c for c in query.split("celsius")[0] if c.isdigit() or c == '.'))
            return f"{val}C = {(val * 9/5 + 32):.2f}F"
        if "fahrenheit" in query and "celsius" in query:
            val = float(''.join(c for c in query.split("fahrenheit")[0] if c.isdigit() or c == '.'))
            return f"{val}F = {(val - 32) * 5/9:.2f}C"

        if "km" in query and "miles" in query:
            val = float(''.join(c for c in query.split("km")[0] if c.isdigit() or c == '.'))
            return f"{val} km = {(val * 0.6214):.4f} miles"
        if "miles" in query and "km" in query:
            val = float(''.join(c for c in query.split("miles")[0] if c.isdigit() or c == '.'))
            return f"{val} miles = {(val * 1.6093):.4f} km"

        if "kg" in query and ("lb" in query or "pound" in query):
            val = float(''.join(c for c in query.split("kg")[0] if c.isdigit() or c == '.'))
            return f"{val} kg = {(val * 2.2046):.4f} pounds"

        return "Unit conversion not recognised. Try: '100 km to miles', '37 celsius to fahrenheit',
'70 kg to pounds'"

    except Exception:
        return "Could not parse unit conversion query."

# -----

```

```
# STEP 3: Decision Node -- LLM Picks the Right Tool
# -----

def decide_tool(state: AgentState) -> AgentState:
    """
    LLM reads the query and decides which tool to use.
    Returns a JSON object with tool_name and tool_input.
    """
    prompt = f"""You are a smart assistant with access to these tools:

1. calculator -- for any math: arithmetic, algebra, geometry formulas
2. weather -- for weather queries about a city
3. unit_converter -- for converting km/miles, celsius/fahrenheit, kg/pounds
4. none -- for general questions, explanations, or greetings

User query: {state['query']}

Respond with ONLY a valid JSON object, no explanation, no markdown:
{{"tool": "calculator", "input": "876 * 54"}}
{{"tool": "weather", "input": "Chennai"}}
{{"tool": "unit_converter", "input": "100 km to miles"}}
{{"tool": "none", "input": ""}}
"""

    result = llm.invoke(prompt)
    raw = result.content.strip()
    raw = raw.replace("`json", "").replace("`", "").strip()

    try:
        parsed = json.loads(raw)
        state["tool_name"] = parsed.get("tool", "none")
        state["tool_input"] = parsed.get("input", "")
    except json.JSONDecodeError:
        state["tool_name"] = "none"
        state["tool_input"] = ""

    console.print(f"[cyan]LLM chose tool: [bold]{state['tool_name']}[/bold] | input: {state['tool_input']}[/cyan]")
    return state

# -----
# STEP 4: Tool Execution Nodes
# -----

def run_calculator(state: AgentState) -> AgentState:
    console.print(f"[yellow]Calculator: {state['tool_input']}[/yellow]")
    state["tool_result"] = calculator_tool(state["tool_input"])
    return state

def run_weather(state: AgentState) -> AgentState:
    console.print(f"[blue]Weather lookup: {state['tool_input']}[/blue]")
    state["tool_result"] = weather_tool(state["tool_input"])
    return state

def run_unit_converter(state: AgentState) -> AgentState:
    console.print(f"[magenta]Unit converter: {state['tool_input']}[/magenta]")
    state["tool_result"] = unit_converter_tool(state["tool_input"])
    return state

def direct_answer(state: AgentState) -> AgentState:
    """
    No tool needed -- LLM answers directly from its own knowledge.
    """
```

```

"""
console.print(f"[green]Direct LLM answer[/green]")
result = llm.invoke(state["query"])
state["tool_result"] = result.content.strip()
return state

# -----
# STEP 5: Synthesis Node -- LLM Wraps Tool Result into an Answer
# -----

def synthesize(state: AgentState) -> AgentState:
    """
    Takes the tool result and forms a natural language response.
    Skipped if direct_answer already handled it.
    """
    if state["tool_name"] == "none":
        state["final_answer"] = state["tool_result"]
        return state

    prompt = f"""Original question: {state['query']}

    Tool used: {state['tool_name']}
    Tool result: {state['tool_result']}

    Write a clear, helpful answer to the user using the tool result.
    Be concise -- 1 to 3 sentences maximum.
    """
    result = llm.invoke(prompt)
    state["final_answer"] = result.content.strip()
    return state

# -----
# STEP 6: Router for Conditional Edges
# -----

def tool_router(state: AgentState) -> str:
    tool = state["tool_name"]
    if tool == "calculator":
        return "calculator"
    elif tool == "weather":
        return "weather"
    elif tool == "unit_converter":
        return "unit_converter"
    else:
        return "direct"

# -----
# STEP 7: Build the Graph
# -----

builder = StateGraph(AgentState)

builder.add_node("decide_tool", decide_tool)
builder.add_node("calculator", run_calculator)
builder.add_node("weather", run_weather)
builder.add_node("unit_converter", run_unit_converter)
builder.add_node("direct", direct_answer)
builder.add_node("synthesize", synthesize)

builder.add_conditional_edges(
    "decide_tool",
    tool_router,
    {

```

```

        "calculator":      "calculator",
        "weather":        "weather",
        "unit_converter":  "unit_converter",
        "direct":          "direct",
    }
)

builder.add_edge("calculator",      "synthesize")
builder.add_edge("weather",        "synthesize")
builder.add_edge("unit_converter", "synthesize")
builder.add_edge("direct",         "synthesize")

builder.set_entry_point("decide_tool")
builder.set_finish_point("synthesize")

graph = builder.compile()

# -----
# STEP 8: Run Demo Queries
# -----

queries = [
    "What is 876 * 54?",
    "What is sqrt(144) + 25?",
    "What is the weather in Chennai?",
    "How is the weather in Coimbatore today?",
    "Convert 120 km to miles",
    "Convert 37 celsius to fahrenheit",
    "What is the capital of France?",
    "What is 15% of 3500?",
]

if __name__ == "__main__":
    console.print("\n[bold magenta]=== ReAct Agent with Tool Calling ===[/bold magenta]\n")

    for query in queries:
        console.print(f"\n[bold white]Query:[/bold white] {query}")

        result = graph.invoke({
            "query":      query,
            "tool_name":  "",
            "tool_input": "",
            "tool_result": "",
            "final_answer": "",
        })

        console.print(Panel(
            result["final_answer"],
            title="[bold green]Answer[/bold green]",
            border_style="green"
        ))

```

Sample Input / Output

"What is 876 * 54?"

--- Output ---

LLM chose tool: calculator | input: 876 * 54
 Answer: 876 multiplied by 54 equals 47,304.

Key Concepts Covered

Concept	Description
ReAct Pattern	Reason -> Act -> Respond loop for tool-using agents.
Tool Calling	LLM selects a tool; deterministic Python code executes it.
JSON-Constrained Output	LLM is forced to answer in a strict, parseable format.

Lab 5: Memory-Enabled AI Assistant (Session Memory)

Source Code

```
# =====
# LAB 5 -- Memory-Enabled AI Assistant
# =====

from typing import TypedDict, List, Optional
from langgraph.graph import StateGraph
from langchain_ollama import ChatOllama
from rich.console import Console
from rich.panel import Panel
from rich.table import Table
import json

console = Console()
llm = ChatOllama(model="qwen2.5:3b", temperature=0.3)

# -- GLOBAL SESSION MEMORY -----
# This dictionary persists across graph.invoke() calls within a session.
# Lab 6 replaces this with SQLite for multi-session persistence.

session_memory = {
    "user_name": None,
    "user_interests": [],
    "user_preferences": {},
    "extracted_facts": [],
    "conversation_history": [],
}

# -- STATE -----

class MemoryState(TypedDict):
    user_input: str
    user_name: Optional[str]
    user_interests: List[str]
    user_preferences: dict
    extracted_facts: List[str]
    answer: str

# -- NODE 1: Extract Memory -----

def extract_memory(state: MemoryState) -> MemoryState:
    """
    LLM scans the message for personal facts and merges them
    into the global session_memory dictionary.
    """
    prompt = (
        "You are a memory extraction system.\n"
        "Read this message and extract personal facts.\n\n"
        "Message: \"" + state["user_input"] + "\"\n\n"
        "Respond with ONLY valid JSON, no explanation:\n"
        '{"name": null, "interests": [], "preferences": {}, "other_facts": []}'
    )

    result = llm.invoke(prompt)
    raw = result.content.strip().replace("`json", "").replace("`", "").strip()

    try:
        extracted = json.loads(raw)
        if extracted.get("name"):
            session_memory["user_name"] = extracted["name"]
        for item in extracted.get("interests", []):
            if item not in session_memory["user_interests"]:
                session_memory["user_interests"].append(item)
```

```

        session_memory["user_preferences"].update(extracted.get("preferences", {}))
    for fact in extracted.get("other_facts", []):
        if fact not in session_memory["extracted_facts"]:
            session_memory["extracted_facts"].append(fact)
except json.JSONDecodeError:
    pass

state["user_name"] = session_memory["user_name"]
state["user_interests"] = list(session_memory["user_interests"])
state["user_preferences"] = dict(session_memory["user_preferences"])
state["extracted_facts"] = list(session_memory["extracted_facts"])
return state

# -- NODE 2: Generate Personalized Answer -----

def generate_answer(state: MemoryState) -> MemoryState:
    """
    Builds a memory-enriched prompt and generates the response.
    """
    lines = []
    if state["user_name"]:
        lines.append("Name: " + state["user_name"])
    if state["user_interests"]:
        lines.append("Interests: " + ", ".join(state["user_interests"]))
    for k, v in state["user_preferences"].items():
        lines.append(k + ": " + str(v))
    for f in state["extracted_facts"]:
        lines.append("Fact: " + str(f))
    memory_block = "\n".join(lines) if lines else "Nothing known yet."

    history_text = ""
    for turn in session_memory["conversation_history"][-6:]:
        role = "User" if turn["role"] == "user" else "Assistant"
        history_text += role + ": " + turn["text"] + "\n"

    prompt = (
        "You are a friendly personal assistant with a good memory.\n\n"
        "--- WHAT YOU KNOW ABOUT THIS PERSON ---\n"
        + memory_block + "\n\n"
        "--- RECENT CONVERSATION ---\n"
        + history_text + "\n"
        "--- CURRENT MESSAGE ---\n"
        "User: " + state["user_input"] + "\n\n"
        "Instructions:\n"
        "- Use their name naturally if you know it\n"
        "- Reference past info only when genuinely helpful\n"
        "- If they ask what you remember, list everything\n"
        "- Be concise and warm\n\n"
        "Assistant:"
    )

    result = llm.invoke(prompt)
    state["answer"] = result.content.strip()

    session_memory["conversation_history"].append({"role": "user", "text": state["user_input"]})
    session_memory["conversation_history"].append({"role": "assistant", "text": state["answer"]})

    return state

# -- BUILD GRAPH -----

builder = StateGraph(MemoryState)
builder.add_node("extract_memory", extract_memory)
builder.add_node("generate_answer", generate_answer)
builder.add_edge("extract_memory", "generate_answer")

```

```

builder.set_entry_point("extract_memory")
builder.set_finish_point("generate_answer")
graph = builder.compile()

# -- HELPER: Show Memory Table -----

def show_memory():
    table = Table(title="Current Memory State", style="bold cyan")
    table.add_column("Field", style="cyan", width=20)
    table.add_column("Value", style="white", width=50)
    table.add_row("Name", str(session_memory["user_name"] or "-"))
    table.add_row("Interests", ", ".join(session_memory["user_interests"]) or "-")
    table.add_row("Preferences", str(session_memory["user_preferences"]) or "-")
    table.add_row("Other Facts", str(session_memory["extracted_facts"]) or "-")
    table.add_row("Turns", str(len(session_memory["conversation_history"]) // 2))
    console.print(table)

# -- MAIN CHAT LOOP -----

if __name__ == "__main__":
    console.print("\n[bold magenta]=== Memory-Enabled AI Assistant ===[/bold magenta]")
    console.print("[dim]Commands: memory | clear | quit[/dim]\n")

    while True:
        user_input = console.input("[bold cyan]You: [/bold cyan]").strip()
        if not user_input:
            continue
        if user_input.lower() == "quit":
            console.print("[yellow]Goodbye![/yellow]")
            break
        if user_input.lower() == "memory":
            show_memory()
            continue
        if user_input.lower() == "clear":
            session_memory.update({"user_name": None, "user_interests": [], "user_preferences": {},
"extracted_facts": [], "conversation_history": []})
            console.print("[yellow]Memory cleared.[/yellow]")
            continue

        result = graph.invoke({
            "user_input": user_input, "user_name": None,
            "user_interests": [], "user_preferences": {},
            "extracted_facts": [], "answer": "",
        })

        console.print(Panel(result["answer"], title="[bold green]Assistant[/bold green]",
border_style="green"))

```

Sample Input / Output

You: Hi, I'm Aarav and I love cricket and AI

--- Output ---

Assistant: Nice to meet you, Aarav! Cricket and AI – great combination. How can I help today?

Key Concepts Covered

Concept	Description
Session Memory	An in-process dict that persists across invoke() calls.
Memory Extraction	Using the LLM itself to turn chat into structured facts.
Memory-Augmented Prompting	Known facts are injected into the prompt before generation.

Lab 6: Persistent Memory with SQLite Checkpointing

Source Code

```
# =====
# LAB 6 (ADVANCED) -- Persistent Memory with SQLite Checkpointing
# =====

from typing import TypedDict, List, Annotated
from langgraph.graph import StateGraph
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, AIMessage
from langgraph.graph.message import add_messages
from rich.console import Console
from rich.panel import Panel
import sqlite3
import os

console = Console()
llm = ChatOllama(model="qwen2.5:3b", temperature=0.3)

DB_PATH = "memory.db"

# -- STATE WITH MESSAGE ACCUMULATOR -----

class ChatState(TypedDict):
    messages: Annotated[List, add_messages]

# -- NODE: Chat Response -----

def chat_node(state: ChatState) -> ChatState:
    """
    Receives the full message history, generates a response.
    The checkpointer saves the updated state to SQLite automatically.
    """
    system = (
        "You are a helpful, friendly AI assistant with perfect memory. "
        "You remember everything the user has told you in this conversation. "
        "Refer to past information naturally when relevant."
    )

    history_text = ""
    for msg in state["messages"][:-1]: # Exclude the latest (current) message
        if hasattr(msg, "content"):
            role = "User" if msg.__class__.__name__ == "HumanMessage" else "Assistant"
            history_text += role + ": " + msg.content + "\n"

    current_message = state["messages"][-1].content

    prompt = (
        system + "\n\n"
        "Conversation so far:\n" + history_text + "\n"
        "User: " + current_message + "\n\n"
        "Assistant:"
    )

    result = llm.invoke(prompt)

    return {"messages": [AIMessage(content=result.content.strip())]}

# -- BUILD GRAPH WITH SQLITE CHECKPOINTER -----
```

```

def build_graph_with_memory(db_path: str):
    """
    Creates the graph and attaches a SQLite checkpoint.
    The checkpoint intercepts every state update and saves it to disk.
    """
    builder = StateGraph(ChatState)
    builder.add_node("chat", chat_node)
    builder.set_entry_point("chat")
    builder.set_finish_point("chat")

    # SqliteSaver connects to (or creates) the SQLite database
    conn = sqlite3.connect(db_path, check_same_thread=False)
    memory = SqliteSaver(conn)

    # compile(checkpointer=memory) activates persistence
    graph = builder.compile(checkpointer=memory)
    return graph

# -- HELPER: Show Chat History for a Thread -----

def show_history(db_path: str, thread_id: str):
    """
    Directly reads from SQLite to show stored conversation history.
    """
    if not os.path.exists(db_path):
        console.print("[yellow]No database found yet.[/yellow]")
        return

    try:
        conn = sqlite3.connect(db_path)
        cursor = conn.cursor()
        cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
        tables = [row[0] for row in cursor.fetchall()]
        console.print(f"[dim]Database tables: {tables}[/dim]")
        conn.close()
    except Exception as e:
        console.print(f"[red]DB read error: {e}[/red]")

# -- MAIN CHAT LOOP -----

if __name__ == "__main__":
    console.print("\n[bold magenta]=== Persistent Memory AI (SQLite) ===[/bold magenta]")
    console.print(f"[dim]Database: {os.path.abspath(DB_PATH)}[/dim]")

    console.print("\n[cyan]Enter a Thread ID (press Enter for default 'user_001'):[/cyan]")
    thread_id_input = console.input("[bold cyan]Thread ID: [/bold cyan]").strip()
    thread_id = thread_id_input if thread_id_input else "user_001"

    console.print(f"\n[green]Using thread: [bold]{thread_id}[/bold][[/green]")
    console.print(f"[dim]Type 'quit' to exit. Your conversation is saved automatically.[/dim]\n")

    graph = build_graph_with_memory(DB_PATH)

    # Config dictionary tells LangGraph which thread's checkpoint to load/save
    config = {"configurable": {"thread_id": thread_id}}

    while True:
        user_input = console.input("[bold cyan]You: [/bold cyan]").strip()
        if not user_input:
            continue
        if user_input.lower() == "quit":
            console.print(f"[yellow]Conversation saved to {DB_PATH}. See you next time![/yellow]")
            break

```

```

    result = graph.invoke(
        {"messages": [HumanMessage(content=user_input)]},
        config=config
    )

    last_message = result["messages"][-1]
    console.print(Panel(
        last_message.content,
        title=f"[bold green]Assistant (thread: {thread_id})[/bold green]",
        border_style="green"
    ))

    console.print(f"\n[dim]Memory file location: {os.path.abspath(DB_PATH)}[/dim]")

```

Sample Input / Output

```

Thread ID: user_001
You: My favorite color is blue.

--- Output ---
Assistant (thread: user_001): Got it – blue is your
favorite color!
[after restart, same thread_id remembers this fact]

```

Key Concepts Covered

Concept	Description
SqliteSaver	Persists/restores full graph state to/from SQLite, keyed by thread_id.
add_messages Reducer	Makes a state field append instead of overwrite.
Durable Memory	State survives process restarts, unlike Lab 5.

Lab 7: Capstone — Multi-Agent University Assistant

Source Code

```
# =====
# CAPSTONE -- Multi-Agent University Assistant
# =====

from typing import TypedDict, Optional, List, Annotated
from langgraph.graph import StateGraph
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, AIMessage
from langgraph.graph.message import add_messages
from rich.console import Console
from rich.panel import Panel
from rich.table import Table
import json
import os
import sqlite3

console = Console()
llm = ChatOllama(model="qwen2.5:3b", temperature=0.3)

DB_PATH = "university_memory.db"

# -- UNIVERSITY KNOWLEDGE BASE -----

ADMISSION_KB = {
    "btech_fees": "B.Tech fees: Rs 1,50,000 per year. Hostel: Rs 60,000 per year.",
    "eligibility": "Minimum 60% in Class 12 with Physics, Chemistry, Maths.",
    "courses": "B.Tech: CSE, AI&DS, ECE, Mechanical, Civil. M.Tech: AI, VLSI, Robotics.",
    "application": "Apply at admissions.vit.ac.in. Last date: March 31.",
    "entrance": "VITEEE conducted every April. 125 questions, 2.5 hours.",
}

HOSTEL_KB = {
    "availability": "Boys: 4000 seats. Girls: 3500 seats. Apply early.",
    "room_types": "Single: Rs 80,000/yr. Double: Rs 60,000/yr. Triple: Rs 50,000/yr.",
    "rules": "Curfew: 10 PM. No visitors after 8 PM. Ragging strictly prohibited.",
    "mess": "Veg and non-veg options. Meal timings: 7AM, 1PM, 7PM. Monthly pass: Rs 3,500.",
    "facilities": "24x7 Wi-Fi, gym, common room, laundry service, medical center.",
}

PLACEMENT_KB = {
    "companies": "Microsoft, Amazon, Zoho, TCS, Infosys, Wipro, Cognizant, Accenture.",
    "packages": "Average: 6.5 LPA. Highest 2024: 42 LPA (Microsoft). Median: 5.8 LPA.",
    "internships": "Summer internships: Dec-Jan registrations. Companies: 200+.",
    "eligibility": "CGPA above 6.0 required for most companies. No active backlogs.",
    "preparation": "Training provided: aptitude, coding, communication. Starts Semester 5.",
}

ACADEMIC_KB = {
    "attendance": "Minimum 75% attendance required. Below 75%: not allowed in exams.",
    "credits": "B.Tech total: 160 credits. Each subject: 3-4 credits.",
    "semester": "2 semesters per year. Odd: July-Nov. Even: Jan-May. Summer: optional.",
    "grading": "GPA out of 10. Above 9: S grade. 8-9: A. 7-8: B. Pass: 5 and above.",
    "backlog": "Failed subjects can be cleared in supplementary exams or re-registration.",
}

# -- STATE -----

class UniversityState(TypedDict):
    messages: Annotated[List, add_messages]
    student_name: Optional[str]
```

```

register_no: Optional[str]
department: Optional[str]
cgpa: Optional[str]
query_category: str
kb_context: str

# -- NODE 1: Student Profile Extractor -----

def extract_profile(state: UniversityState) -> UniversityState:
    """
    Detects if the user mentioned their name, register number, or department.
    Updates state fields if found.
    """
    last_msg = state["messages"][-1].content

    prompt = (
        "Extract student profile info from this message.\n"
        "Message: \"" + last_msg + "\"\n\n"
        "Respond with ONLY valid JSON:\n"
        "{\"name\": null, \"register_no\": null, \"department\": null, \"cgpa\": null}"
    )

    result = llm.invoke(prompt)
    raw = result.content.strip().replace("```json", "").replace("```", "").strip()

    try:
        extracted = json.loads(raw)
        if extracted.get("name") and not state.get("student_name"):
            state["student_name"] = extracted["name"]
        if extracted.get("register_no") and not state.get("register_no"):
            state["register_no"] = extracted["register_no"]
        if extracted.get("department") and not state.get("department"):
            state["department"] = extracted["department"]
        if extracted.get("cgpa") and not state.get("cgpa"):
            state["cgpa"] = str(extracted["cgpa"])
    except json.JSONDecodeError:
        pass

    return state

# -- NODE 2: Router -- Classify Query -----

def classify_query(state: UniversityState) -> UniversityState:
    """
    Classifies the query into one of 4 domains.
    """
    last_msg = state["messages"][-1].content

    prompt = (
        "Classify this university student query into ONE category:\n"
        "admission, hostel, placement, academic\n\n"
        "Query: " + last_msg + "\n\n"
        "Rules:\n"
        "- admission: fees, courses, eligibility, application, entrance exam\n"
        "- hostel: rooms, mess, curfew, facilities, hostel rules\n"
        "- placement: jobs, companies, salary, packages, internships\n"
        "- academic: attendance, grades, credits, semester, backlog\n\n"
        "Reply with ONLY one word:"
    )

    result = llm.invoke(prompt)
    category = result.content.strip().lower()

    valid = {"admission", "hostel", "placement", "academic"}
    state["query_category"] = category if category in valid else "admission"

```

```

console.print(f"[cyan]Routed to: [bold]{state['query_category'].upper()} AGENT[/bold][[/cyan]]")
return state

# -- NODE 3: Agent Nodes -- Each handles one domain -----

def _agent_respond(state: UniversityState, knowledge_base: dict, domain_name: str) -> UniversityState:
    """
    Generic agent: retrieves relevant KB entries, builds a prompt, generates response.
    """
    last_msg = state["messages"][-1].content

    # Simple keyword search across KB
    relevant_entries = []
    for key, value in knowledge_base.items():
        if any(word in last_msg.lower() for word in key.split("_")):
            relevant_entries.append(value)

    # Fallback: include all KB if nothing matched
    if not relevant_entries:
        relevant_entries = list(knowledge_base.values())

    context = "\n".join(relevant_entries[:4])
    state["kb_context"] = context

    student_info = ""
    if state.get("student_name"):
        student_info += "Student name: " + state["student_name"] + "\n"
    if state.get("department"):
        student_info += "Department: " + state["department"] + "\n"
    if state.get("cgpa"):
        student_info += "CGPA: " + state["cgpa"] + "\n"

    prompt = (
        "You are the " + domain_name + " specialist at VIT University.\n\n"
        "Student information:\n" + (student_info if student_info else "Unknown\n") + "\n"
        "Relevant university information:\n" + context + "\n\n"
        "Student question: " + last_msg + "\n\n"
        "Answer helpfully in 3-5 sentences. Use student's name if known. "
        "Be specific with numbers and facts from the information above.\n\n"
        "Answer:"
    )

    result = llm.invoke(prompt)
    return {**state, "messages": [AIMessage(content=result.content.strip())]}

def admission_agent(state: UniversityState) -> UniversityState:
    return _agent_respond(state, ADMISSION_KB, "Admissions")

def hostel_agent(state: UniversityState) -> UniversityState:
    return _agent_respond(state, HOSTEL_KB, "Hostel Administration")

def placement_agent(state: UniversityState) -> UniversityState:
    return _agent_respond(state, PLACEMENT_KB, "Placement Cell")

def academic_agent(state: UniversityState) -> UniversityState:
    return _agent_respond(state, ACADEMIC_KB, "Academic Affairs")

# -- ROUTER FUNCTION -----

def agent_router(state: UniversityState) -> str:
    return state["query_category"]

```

```
# -- BUILD GRAPH -----

def build_university_graph(db_path: str):
    builder = StateGraph(UniversityState)

    builder.add_node("extract_profile", extract_profile)
    builder.add_node("classify_query", classify_query)
    builder.add_node("admission", admission_agent)
    builder.add_node("hostel", hostel_agent)
    builder.add_node("placement", placement_agent)
    builder.add_node("academic", academic_agent)

    builder.add_edge("extract_profile", "classify_query")

    builder.add_conditional_edges(
        "classify_query",
        agent_router,
        {
            "admission": "admission",
            "hostel": "hostel",
            "placement": "placement",
            "academic": "academic",
        }
    )

    builder.set_entry_point("extract_profile")
    builder.set_finish_point("admission")
    builder.set_finish_point("hostel")
    builder.set_finish_point("placement")
    builder.set_finish_point("academic")

    conn = sqlite3.connect(db_path, check_same_thread=False)
    memory = SqliteSaver(conn)
    return builder.compile(checkpointer=memory)

# -- MAIN -----

if __name__ == "__main__":
    console.print("\n[bold magenta]=== VIT University Multi-Agent Assistant ===[/bold magenta]")
    console.print(f"[dim]Persistent memory: {os.path.abspath(DB_PATH)}[/dim]\n")

    console.print("[cyan]Enter your student ID or name as a session key:[/cyan]")
    thread_id = console.input("[bold cyan]Session ID: [/bold cyan]").strip() or "student_001"

    console.print(f"\n[green]Session: [bold]{thread_id}[/bold] | Type 'quit' to exit[/green]\n")

    graph = build_university_graph(DB_PATH)
    config = {"configurable": {"thread_id": thread_id}}

    table = Table(title="Available Agents", style="bold")
    table.add_column("Agent", style="cyan", width=20)
    table.add_column("Handles", style="white", width=50)
    table.add_row("Admissions", "Fees, courses, eligibility, entrance exam, application")
    table.add_row("Hostel", "Room availability, mess, curfew, facilities, rules")
    table.add_row("Placement", "Companies, salary packages, internships, eligibility")
    table.add_row("Academic", "Attendance, GPA, credits, semester, backlog")
    console.print(table)
    console.print()

    while True:
        user_input = console.input("[bold cyan]You: [/bold cyan]").strip()
        if not user_input:
            continue
```

```

if user_input.lower() == "quit":
    console.print("[yellow]Goodbye! Your conversation is saved.[/yellow]")
    break

result = graph.invoke(
    {"messages": [HumanMessage(content=user_input)]},
    config=config
)

last_msg = result["messages"][-1]
category = result.get("query_category", "").upper()

console.print(Panel(
    last_msg.content,
    title=f"[bold green]{category} AGENT[/bold green]",
    border_style="green"
))

```

Sample Input / Output

You: Hi I'm Priya, CGPA 8.2, CSE. What is the hostel curfew?

--- Output ---

Routed to: HOSTEL AGENT

Hi Priya! The hostel curfew is 10 PM, and no visitors are allowed after 8 PM.

Key Concepts Covered

Concept	Description
Multi-Agent Routing	One router, four domain specialists sharing the same graph.
Knowledge Grounding (RAG-style)	Keyword-matched facts are injected into each agent's prompt.
End-to-End Persistence	Profile + conversation both survive across sessions.

Cheat Sheet

Syntax / Call	Purpose
<code>StateGraph(StateSchema)</code>	Create a new graph builder bound to a TypedDict state schema.
<code>builder.add_node(name, fn)</code>	Register a node function under a name.
<code>builder.add_edge(a, b)</code>	Always go from node a to node b after a finishes.
<code>builder.add_conditional_edges(src, router_fn, mapping)</code>	Branch from src based on router_fn's return value.
<code>builder.set_entry_point(name)</code>	Mark the node where graph execution begins.
<code>builder.set_finish_point(name)</code>	Mark a node where graph execution may terminate.
<code>builder.compile(checkpointer=...)</code>	Lock the graph into a runnable object; optionally enable persistence.
<code>graph.invoke(state, config=...)</code>	Run the graph once; config carries thread_id for persistent graphs.
<code>Annotated[list, add_messages]</code>	State field type that appends new messages instead of overwriting.
<code>SqliteSaver(sqlite3.connect(...))</code>	Checkpointers that persists/restores state to/from a SQLite file.
END	Special LangGraph sentinel used to explicitly terminate a graph from a router.

Industry Use Cases

Pattern	Real-World Application
Single-node graph	Templated document/notification generation pipelines.
LLM classification + routing	Customer support ticket triage, helpdesk automation.
Sequential pipeline	Loan/insurance application processing, onboarding workflows.
ReAct + tool calling	AI coding assistants, finance and travel-booking agents.
Session / persistent memory	Personal AI companions, long-running customer-service chat history.
Multi-agent + knowledge grounding	Enterprise IT/HR helpdesks, multi-department virtual receptionists.

Troubleshooting

Issue	Solution
Model not responding (ChatOllama)	Ensure Ollama is running and 'ollama pull qwen2.5:3b' was run.
JSON parse errors in classify/extract nodes	Strip markdown fences; tighten the prompt's 'ONLY JSON' instruction.
Category/tool defaults unexpectedly	Lower temperature to 0; add clearer category examples in the prompt.
KeyError on a state field	Initialise every TypedDict field with a default value at invoke().
sqlite3 'database is locked'	Avoid parallel writers to the same thread_id.
Conditional edge 'node not found'	Ensure the router's return values all exist in the edge mapping.

Conclusion

Across these seven experiments, students progressed from a single-node LangGraph program to a persistent, multi-agent university assistant — covering state management, conditional routing, LLM classification, tool calling, memory, and multi-agent orchestration. Built entirely on a free local model via Ollama, this curriculum gives students hands-on, code-first experience they can directly apply to real agentic AI projects.